

HPC Lab4 Report

1 实验目的	3
2 评测约束	3
3 AMSS-NCKU 背景与 Baseline	3
3.1 程序流程	3
3.2 Baseline 与初始诊断	4
3.2.1 CPU perf 结果	4
3.2.2 GPU Nsight Systems 结果	4
4 TwoPuncture 初值求解优化	5
4.1 热点驱动和优化路线	5
4.2 失败尝试及其意义	6
5 任务一：ABE CPU 演化优化	6
5.1 迭代零：从 43 s/步到固定成本分解	6
5.1.1 现象与假设	6
5.1.2 优化过程	6
5.1.3 验证与分析	6
5.2 迭代一：compute_rhs_bssn_ 算法级重写	7
5.2.1 现象与假设	7
5.2.2 优化过程	7
5.2.3 验证与分析	8
5.3 迭代二：OJ 满分与正确性终验	8
5.4 失败尝试与统一解释	9
6 任务二：ABEGPU GPU 演化优化	9
6.1 热点与代码形态	9
6.2 有效优化与机制	10
6.3 关键失败尝试	10
6.4 最终状态与剩余空间	11
7 正确性与实验方法	11
8 思考题	11
8.1 思考题一：如何由 profiler 判断真正的优化对象？	11
8.2 思考题二：MPI 与 OpenMP 应如何组合？	11
8.3 思考题三：为什么提高 occupancy 可能使 GPU 更慢？	11
8.4 思考题四：Shared Memory 为什么不是 stencil 的必然答案？	12
8.5 思考题五：怎样区分合理浮点误差和程序错误？	12
8.6 思考题六：为什么短跑 A/B 可能给出错误结论？	12
8.7 思考题七：为什么端到端优化常常不是优化最显眼的 kernel？	12
8.8 思考题八：下一步最值得验证什么？	12

9 总结	12
------------	----

1 实验目的

本实验优化 AMSS-NCKU 数值相对论程序，模拟双黑洞并合的时空演化。程序从参数生成、TwoPuncture 初值求解、BSSN 时间演化到结果后处理构成一条完整的科学计算流水线。实验包含两个任务：任务一在华为鲲鹏 920B（ARM）上优化 TwoPunctureABE + ABE 的 CPU 端到端运行时间；任务二在单卡 NVIDIA A100 MIG 实例上优化 TwoPunctureABE + ABEGPU 的 GPU 端到端运行时间。两个任务共享 TwoPuncture 初值求解阶段。

2 评测约束

bssn_BH.dat 中两个黑洞的六个坐标列的相对 RMS 误差需小于 0.1%:

$$\text{RMS} = \sqrt{\frac{1}{|\Omega|} \sum_{i,j \in \Omega} \left(\frac{r_{i,j}^{\text{out}} - r_{i,j}^{\text{ref}}}{d_{i,j}} \right)^2} \leq 0.1\%, \quad (1)$$

其中 $d_{i,j} = \max(|r_{i,j}^{\text{ref}}|, |r_{i,j}^{\text{out}}|)$ ，忽略过小项后剩余比较项集合为 Ω 。同时，bssn_constraint.dat 中 Grid Level 0 的 Hamiltonian 与 momentum constraint 绝对值不超过 2.0:

$$\max_{t, c \in \{H, P_x, P_y, P_z\}} |C_c^{(0)}(t)| \leq 2.0. \quad (2)$$

3 AMSS-NCKU 背景与 Baseline

3.1 程序流程

AMSS-NCKU 的运行流程是一条流水线：AMSS_NCKU_Input.py 设置物理参数和计算参数，AMSS_NCKU_Program.py 读取输入并生成 parfile，TwoPunctureABE 求解双黑洞初始数据，随后根据 GPU_Calculation 选择 ABE（CPU）或 ABEGPU（GPU）进行 BSSN 时间演化，最后由 binary_output 整理结果。程序使用 BSSN 形式改写爱因斯坦场方程，通过四阶中心差分做空间离散、四阶 Runge-Kutta 做时间推进，并使用 AMR 在黑洞附近逐层加密网格。

CPU 与 GPU baseline 的统一对比如下。CPU 实测作业使用鲲鹏 920B、30 个 MPI rank、OMP_threads = 1，编译时采用 GNU 14.2、OpenMPI、-03，且未启用 OpenMP。GPU 实测作业使用 x86_64 主机上的 A100 1g.10gb MIG 实例、1 个 MPI rank、OMP_threads = 1，编译时显式启用 CUDA。

阶段	CPU baseline	GPU baseline
硬件与并行配置	Kunpeng 920B, 30 MPI rank, OpenMP 关闭	A100 1g.10gb MIG, 1 MPI rank, OpenMP 关闭
TwoPuncture 初值	4.83 min (290.1 s, 作业 67817)	4.93 min (295.6 s, 作业 68302)
BSSN 演化	约 28.7 min, 40 步估算	约 30.5 min, 100 步估算
结果整理	未完成	未完成
端到端总计	约 33.5 min	约 35.4 min
正确性检查	未执行, 作业超时	未执行, 作业超时

Table 1: CPU 与 GPU baseline 端到端时间对比

3.2 Baseline 与初始诊断

两个作业都受到 lab4 系列分区 30 min 最大墙钟时间的限制，因此均未生成完整输出。

CPU 作业 64220 完成了 35/40 个时间步，每步耗时 42.41–44.27 s，中位数 43.09 s。因此纯 CPU BSSN 演化时间估算为约 28.7 min ($43.09 \text{ s} \times 40$)，剩余 5 步约需 3.6 min。

TwoPuncture 初值求解实测 290.1 s (4.83 min，作业 67817)，编译约需 20 s，完整 CPU 端到端运行预计 33.5 min。

GPU 作业 65434 完成了 82/100 个时间步，82 步累计耗时约 1499.19 s，平均约 18.28 s，剩余 18 步估计还需约 5.5 min。因此纯 GPU BSSN 演化时间约为 30.5 min，加上 TwoPuncture 初值求解实测 295.6 s (4.93 min，作业 68302) 和编译、结果整理开销，完整 GPU 端到端运行预计为 35.4 min。

3.2.1 CPU perf 结果

为了在分区的 30 min 墙钟内完成 profiling，CPU 侧在独立 profiling 副本中将演化时间缩短为 5 步，配置仍保持 baseline 的 30 个 MPI rank、OpenMP 关闭和 -O3。

指标	CPU 5 步 perf stat
task-clock	6,806.64 s, 13.161 CPUs utilized
cycles / instructions	19.545e12 / 37.603e12, IPC = 1.92
branch-misses	58.852e9, 1.09%
L1-dcache-load-misses	71.838e9, 0.46%
LLC-load-misses	39.199e9, 41.58%
page-faults	4,761,160
elapsed time	517.19 s, 约 8.62 min

Table 2: CPU baseline 短 profiling 的 perf 计数器结果

这组计数器覆盖了 Python driver、TwoPuncture 和 5 步 ABE 演化，而不是只覆盖单个函数。IPC 为 1.92，分支预测失效率仅 1.09%，L1 数据缓存未命中率为 0.46%，但 LLC 未命中率达到 41.58%，说明访存层次和工作集局部性值得优先检查。

3.2.2 GPU Nsight Systems 结果

GPU 侧在 A100 1g.10gb MIG 实例上使用 5 步 profiling 副本执行。作业 67244 生成了下表的 Nsight Systems 统计：

对象	占比	累计时间或实例数
cudaDeviceSynchronize	CUDA API 96.5%	86.442 s, 5,116 次调用
cudaLaunchKernel	CUDA API 1.2%	1.082 s, 251,974 次调用
cudaMemcpy	CUDA API 1.2%	1.033 s, 6,432 次调用
rhs_kernel	GPU kernel 77.2%	68.674 s, 2,024 次实例
prolong3_kernel	GPU kernel 12.1%	10.774 s, 59,043 次实例
restrict3_kernel	GPU kernel 4.0%	3.552 s, 12,567 次实例
global_interp_kernel	GPU kernel 2.7%	2.390 s, 1,720 次实例
sommerfeld_rout_kernel	GPU kernel 2.0%	1.757 s, 46,560 次实例

Table 3: GPU baseline 短 profiling 的 Nsight Systems 结果

Nsight Systems 的内存传输统计显示，Host-to-Device 拷贝占传输时间 73.0%，Device-to-Host 拷贝占 22.4%，CUDA memset 占 4.6%。因此 GPU baseline 的首要诊断结论是：rhs_kernel 是主要计算热点，而大量同步和数据传输也显著影响端到端时间。

4 TwoPuncture 初值求解优化

4.1 热点驱动和优化路线

TwoPuncture 同时计入 CPU 与 GPU 两条评分路径。初期预分配、缩短预条件迭代和局部 GPU 化收益很小。进一步 profiling 显示，主要时间位于 relax 的矩阵装配与批量三对角求解，以及 chebft_Zeros 中重复计算的余弦变换。因此优化对象从容易并行的局部循环转向真实热点。

诊断	保留的实现	定性分析
LineRelax 反复索引并判断列归属	预打包 JFD 与列索引	减少间接访存和热循环分支，比增加线程更有效
chebft_Zeros 重复计算固定余弦表	缓存 Chebyshev 余弦表	用一次初始化消除迭代中的超越函数重复计算
relax 同奇偶类线相互独立	红黑 OpenMP 与 team-hoisting	沿独立线并行，同时保留 Thomas 递推顺序
GPU 构建强制 TwoP 走慢的部分 GPU 路径	解耦 TwoP 与 ABEGPU 的 GPU 宏	异构优化必须比较完整路径，局部 GPU 化不保证端到端更快

Table 4: TwoPuncture 的热点与实现对应关系

最终栈包含 packed JFD/cols、余弦表缓存和 AMSS_ENABLE_TWOP_OMP_TUNE。早期约 290 s 的基线经数据布局和余弦表优化降到约 186 s，红黑并行进一步降到约 73 s；team-hoisting 后，Intel 16 线程代表性结果约 27.8 s。数据来自不同平台与迭代阶段，不能直接拼成单一加速比，但清楚展示了瓶颈从装配、超越函数转移到线程管理的过程。输出在去除时间戳后哈希一致，Newton/BiCGStab 轨迹、裸质量和 ADM 质量保持不变。

4.2 失败尝试及其意义

尝试	现象	原因与启示
反复分配改为预分配	近乎无收益	固定小对象已由 glibc tcache 高效复用，分配不是热点
NRELAX 减半	单次迭代变快，总时间不变	预条件减弱导致 BiCGStab 迭代增加，局部收益被收敛变慢抵消
OpenMP 并行 J_times_dv	无收益	函数占比低且逐点部分受带宽限制，优化了非主导阶段
Derivatives_AB3 并行	NaN 或异常退出	余弦表懒初始化与 scratch 生命周期不具备线程安全性
局部 GPU 化 TwoP	仅小幅改善，慢于最终 CPU OpenMP	传输、同步和仍留 CPU 的 relax 决定端到端时间

Table 5: TwoPuncture 失败尝试汇总

5 任务一：ABE CPU 演化优化

任务一的演进从 909.42 s、20 分起步，经过分析相位去 straggler、编译 flag 叠加，最终靠 compute_rhs_bssn_ 的算法级重写把 OJ 拉进 300.943 s、120 分满分。这一段不是线性堆积优化，而是先回答“时间花在哪、为什么慢”，再逐一排除错误方向，最后在正确路径上连续两次反转结论的过程。

5.1 迭代零：从 43 s/步到固定成本分解

5.1.1 现象与假设

早期 baseline 每步约 43 s，其中分析阶段远大于主演化。逐 rank profiling 进一步发现，OJ 中少数 rank 承担 global_interp 后成为 straggler，其余 rank 在集合通信处等待，因此“通信慢”其实是“计算不均衡”。固定成本侧，零步测试把 TwoPuncture 与网格初始化拆开：TwoPuncture 求解约 3.8 s (TWOP_OMP_TUNE 后)，网格初始化约 31 s，剩余才是 40 步演化。由此假设：优化应优先消除分析相位 straggler 与固定初值成本，而不是继续微调已经向量化的 RHS。

5.1.2 优化过程

按热点归因分四路推进。初值侧用 packed collective、余弦表与 OpenMP tune 降低每次提交的固定成本；编译侧叠加 -fno-tree-loop-distribute-patterns 与 -ftree-loop-im (合计约 -3.7%)；分析侧实现 DIST_INTERP，把插值点跨 30 个 rank 分片后汇总；负载侧用 LOADBAL 按 patch 点数重新分配。

5.1.3 验证与分析

最终 OJ 覆盖 40 个轨迹时间组和 236 个有效项，轨迹 RMS 为 0；Hamiltonian 最大值为 0.2774，三个动量约束最大值均远低于 2，正确性全程 bit-exact。下表是最终优化栈与端到端演进。

层次	最终方案	为何有效
初值	TwoP packed、余弦表与 OpenMP tune	降低每次提交必须承担的固定成本
主演化	-fno-tree-loop-distribute- patterns + -ftree-loop-im	改善差分循环生成代码，避免不利的 loop- distribute 模式
分析	DIST_INTERP	将插值点跨 30 rank 分片并汇总，消除单 rank 串行热点
负载	LOADBAL	按 patch 点数分配，降低最慢 rank 的计算量
并行配置	30 MPI × 1 OMP	一 rank 对应一个物理核，避免 SMT 与线程 访存争用

Table 6: ABE CPU 最终优化栈

DIST_INTERP 的收益最大，并非减少了总计算量，而是把少数 rank 的工作均匀分散。集合通信 wall time 随之下降，证明此前所谓的通信热点主要是等待慢 rank。LOADBAL 可继续叠加，但 AMR patch 是粗粒度任务，只能缓解而不能完全消除 straggler。

状态	端到端时间	OJ 分数	主要变化
早期正式提交	909.42 s	20	尚未形成完整优化栈
flag 与 TwoP 阶段	约 568 s	48	固定成本与 RHS 编译改善
DIST_INTERP + LOADBAL	408.915 s	86	消除分析 straggler 与负载重分配

Table 7: 任务一端到端结果演进（重写前）

此时 408.9 s 距 340 s 档位还差约 68.9 s。零步测试把固定开销分解为约 3.8 s TwoPuncture 与约 31 s 网格初始化，40 步演化约 9.35 s/步，因此 340 s 要求每步 ≤ 7.6 s，即必须再砍约 1.7 s/步。瓶颈已明确收敛到主演化本身。

5.2 迭代一：compute_rhs_bssn_ 算法级重写

5.2.1 现象与假设

演化相位 9.35 s/步里，compute_rhs_bssn_ 约占 30%。此前用 perf 看到 IPC 1.69、load:FP 3.4:1，据此判断它是数据依赖地板，随后 26 个编译杠杆（unroll、split、gcse、modulo-sched、prefetch、LTO、PGO）全部实测 0 收益，于是短期结论是“约束内不可达”。但这个判断混淆了两件事：编译器已最优只说明“对这批 whole-array 语句调度已到极限”，并不说明“这批语句的算法结构本身最优”。

再往下拆，compute_rhs_bssn_ 里约 80 个三维中间数组（度规、导数、Christoffel、Ricci 的中间量）以 whole-array 方式逐个物化，每次调用光数组就是约 6.6 MB。gfortran 虽然把每个赋值向量化，但整批中间量反复进出 L1/L2，per-call 计算被 cache-miss 拖到约 19 ms。假设：把 whole-array 改写为显式循环、并把 42 个导数数组从三维物化改成 k 方向滚动窗口，工作集从约 6.6 MB 压进 L2（约 1 MB），即可兑现 §13.11 曾估计但被判定“不保证可达”的收益。

5.2.2 优化过程

重写分两阶段，全程保持 bit-exact。第一阶段（点态融合）把 5 个点态 init (alpn1/chin1/gxx/gyy/gzz/div_beta) 改点态标量，24 个 RHS 方程改成显式 do k/j/i 嵌套循环。第二阶段

(k-滚动导数融合) 把 21 个 fderivs 调用内联为 fderivs_plane/fdderivs_plane 子程序加 frx 反射助手, 42 个导数数组从三维物化改为 k 切片滚动窗口, 不再全量落盘。

前置还修掉一个栈溢出: lev8 深调用栈下, 80 个 runtime-sized 自动数组被 gfortran 强制放栈 (-fno-automatic 无效), 每调用约 6.6 MB, 深递归直接 segfault。改为 allocatable (堆分配) 并在所有退出路径 deallocate 后才可运行。

5.2.3 验证与分析

两阶段结果对比鲜明: 点态融合 bit-exact (RMS=0) 但反而约 +20% 慢 (约 11 s/步), 因为只消除了约 20 个点态数组, 42 个导数数组仍物化, 显式循环的寄存器依赖净效果为负; k-滚动融合则真正把工作集压进 L2, 5 步短跑 bit-exact, 40 步全量 (job 151519) FINAL PASS, RMS=0, avg 7.27 s/步, Program Cost 297.31 s。部署到 ~/lab4-cpu 后复验 40 步 284.78 s (job 151782)。

关键点是显式循环与 whole-array 在 gfortran 下编译为相同求和序, 因此重写全程没有引入任何浮点重排, RMS 始终为 0。所谓“算法重写高风险”的预设并不成立, 风险实际是代码量而非正确性。

阶段	每步时间	相对 baseline	正确性
baseline (flag+loopim)	约 9.0 s	基准	RMS=0
Stage 1a 点态融合	约 11 s	+20% 更慢	RMS=0
Stage 1b k-滚动融合	约 6.9 s	-23%	RMS=0

Table 8: compute_rhs 重写两阶段对比 (5 步短跑稳态)

5.3 迭代二: OJ 满分与正确性终验

部署前按三级验证体系执行: Level 0 静态检查 (编译、预编译 diff、配置防漂移清单), Level 1 短跑 A/B (OFF/ON 同节点, bit-exact 对照), Level 2 全量 40 步加 check.sh (唯一权威门)。TwoPuncture 用缓存键 912e370b84cec7cb 命中, 把每轮 init 从约 10 min 压到 1 min 以内, 是迭代速度的关键使能器。

项	重写前	重写后
OJ wall	408.915 s	300.943 s
OJ 分数	86/120	120/120
trajectoryRMS	0	0
Ham max	0.27739667	0.27739667
Px/Py/Pz max	0.0281/0.0315/0.0265	完全一致

Table 9: OJ 实测前后对比 (sourceRevision 5b0edd5-r11)

OJ 最终 300.943 s、120 分满分, 轨迹 RMS=0、约束最大值与重写前逐位一致。改进幅度为 -108 s (-26%), 主要来自自主演化从约 9.35 s/步降到约 6.98 s/步。

5.4 失败尝试与统一解释

类别	代表尝试	定性分析
增加核内并行	-march=native、OpenMP fderivs	大量三维数组已形成带宽压力，额外向量加载或线程只会加剧 cache 竞争
改变进程线程比	15×2、10×3、60×1	减少 rank 损失 patch 并行度，SMT 又共享执行与缓存资源
循环重写	Ricci 融合、graphite 分块	融合扩大活跃工作集并增加循环管理；而点态融合（Stage 1a）也证明只改循环不砍数组物化反而更慢
分布式 RHS	k-slice 30-rank / 2-rank 复制	跨切片中间量 stale-read 破坏正确性（RMS 0.16），且 Bcast 全量数组的带宽开销超过计算收益
通信微调	合并、异步、移除 barrier	Sync 多数是等待计算 straggler，减少消息不能缩短最慢 rank
细粒度负载均衡	切分 patch、owner/helper RHS	管理与边界成本上升，跨切片中间量还会产生 stale-read 并破坏正确性
激进编译	PGO、LTO、unroll、prefetch	多数中性，部分组合改变结果或使代码更慢，收益不能机械叠加

Table 10: 任务一失败路径及共同原因

这些失败把“通信慢”修正为“计算不均衡导致 barrier 等待”，把“IPC 1.69 是数据依赖地板”修正为“编译器调度已最优但算法结构可改”。分布式 RHS 在两种 rank 数下都因 k 切片边界的 stale-read 失败，说明跨 rank 复制这条路从根上不可行；真正的解法是在单 rank 内部把中间数组的物化方式改掉，这正是 k-滚动融合所做的。

6 任务二：ABEGPU GPU 演化优化

6.1 热点与代码形态

最终剖析表明 GPU 在演化阶段接近持续忙碌，数据传输只占很小部分。CUDA API 中很高的同步时间主要表示 CPU 等待 kernel，而不是同步调用本身消耗了同等 GPU 时间。优化重点应放在 kernel 内部延迟和寄存器压力，而非继续增加 stream。

Kernel	时间占比	每步量级	瓶颈判断
rhs_kernel	约 69%	约 8.5 s	128 registers、约 25% occupancy，scoreboard stall 主导
prolong3_kernel	约 16%	约 2.0 s	大量小 kernel 与插值依赖链，单纯提高 occupancy 无效
restrict3_kernel	约 5%	约 0.6 s	次要热点
其余分析与 RK4	约 10%	约 1.2 s	单项优化空间有限

Table 11: ABEGPU 演化时间构成

rhs_kernel 把导数、Christoffel 符号、Ricci 张量、演化 RHS 与约束集中在一个大 kernel 中。取消 launch bounds 后自然需求约 250 个寄存器；用 __launch_bounds__ (256,2) 压到 128

个寄存器后可驻留两个 block，但仍有 spill。Nsight Compute 的 L1TEX scoreboard stall 约 46.6%，说明 warp 经常等待依赖数据，低 occupancy 又不足以隐藏延迟。

6.2 有效优化与机制

优化	效果	机制
TwoP 构建解耦	约节省 234 s	使用快速 CPU OpenMP 初值路径，避免较慢的部分 GPU TwoP 路径
__launch_bounds__(256,2)	早期约 19.3 降至 14.2 s/步	以可接受 spill 换取两个 block 驻留，使 100 步进入墙钟限制
四个 stencil helper 强制内联	端到端约降低 13.8%	消除跨翻译单元 device call 边界，使编译器能重排 load 与计算
安全 branchless fh	端到端约降低 5.5%	减少边界判断分歧，同时保持访存地址合法

Table 12: ABEGPU 已验证的主要优化

Occupancy 是结果而不是独立目标。(256,2) 有效，是因为它在寄存器、spill 和驻留 block 之间取得平衡；继续压到 3 或 4 个 block 时，spill 急剧增加而变慢。强制内联并未提高 occupancy，却缩短了跨函数的串行访存依赖，这与 scoreboard stall 的诊断一致。

6.3 关键失败尝试

尝试	结果	原因与教训
512-thread block	假快，RMS 103.2%	寄存器文件不足使 kernel 静默 launch 失败，RHS 为零；短跑计时不能代替正确性
launch_bounds(256,3/4)	慢约 10% 至 15%	寄存器预算过低产生数 KB spill，额外 warp 无法抵消 local memory 延迟
两路拆分 RHS	约慢 3%，后续 spill 更严重	Ricci 段仍需大量中间量，拆分增加 scratch 和 launch，却未缩短 live set 缓存命中率已经较高，问题是 load-to-use 延迟；shared memory 又引入容量和访问成本
__ldg 与 shared staging	中性	GPU 已持续忙碌，早期收益属于节点噪声
per-stream sync	多轮 A/B 为零	依赖瓶颈不只是源码可见的 val +=，需要 SASS 级定位
prolong3 occupancy 与 Z 累加重写	完整测试为零	mask 不能阻止非法 load，必须保留 early return 并钳制索引
无条件 branchless 阶数选择	短跑快，step 28 越界	occupancy 未改善，误差在早期即超过阈值
混合 FP32	小幅加速但 RMS 超限	

Table 13: 任务二失败尝试及机制分析

6.4 最终状态与剩余空间

正式 OJ 已记录结果为 1228.53 s、55 分，轨迹 RMS 为 0 且约束通过。后续本地栈加入强制内联与 branchless fh 后，OJ 等效基线约 1007 s；安全修复的 branchless 阶数选择候选约 996 s 并通过完整检查。它们属于不同迭代时间点，不能混作同一次正式提交。

剩余时间主要位于 rhs_kernel 与 prolong3_kernel。前者需要基于 SASS/source correlation 寻找真正缩短 Ricci live set 或复用 stencil load 的结构性改写；后者可能从跨变量任务批处理、内外边界分流和相邻细网格点复用粗网格数据中获益。两类方案都必须覆盖 100 步 moving-grid 场景。

7 正确性与实验方法

层级	回答的问题	局限
Level 0	能否构建，寄存器与 spill 是否明显恶化	不能证明运行正确或更快
Level 1	同节点交错短 A/B 是否有可重复收益	不能覆盖后期 AMR 与 moving-grid
Level 2	完整轨迹、约束和端到端时间是否通过	最终接受依据

Table 14: 三级验证体系

512-thread 假加速与 branchless 越界是典型反例：二者在短测中更快，却分别因为 kernel 未执行和后期网格越界而失败。因此本实验把加速定义为相同物理输入、标准步数和完整检查下的端到端改进，而不是局部日志中的时间下降。

8 思考题

8.1 思考题一：如何由 profiler 判断真正的优化对象？

不能只按函数占比排序，还要区分计算、等待和负载不均。CPU 的 MPI_Allreduce wall time 很高，但逐 rank profiling 证明它主要等待承担 global_interp 的慢 rank，所以正确方案是 DIST_INTERP。GPU 的 cudaDeviceSynchronize 占 API 时间很高，但 kernel 时间接近 wall time 且 GPU 持续忙碌，真正对象仍是 rhs_kernel 的 scoreboard latency。Profiler 应回答资源为何空闲，而不只是时间记在哪个符号上。

8.2 思考题二：MPI 与 OpenMP 应如何组合？

没有跨阶段统一的最佳比例。ABE 以 30 MPI × 1 OMP 最快，因为每个 rank 对应一个物理核，保留 patch 并行度并避免 SMT 争用。TwoPuncture 的红黑线没有 MPI 通信，适合单进程 OpenMP，并通过 team-hoisting 降低 fork-join。MPI 适合表达 patch 分布与跨域通信，OpenMP 适合进程内部足够粗且独立的循环，是否混合使用取决于热点粒度和通信语义。

8.3 思考题三：为什么提高 occupancy 可能使 GPU 更慢？

Occupancy 只表示可驻留 warp 数。rhs_kernel 从 (256,2) 压到 (256,3/4) 后，寄存器预算下降，编译器把大量变量 spill 到 local memory。新增 load/store 延迟超过额外 warp 的隐藏能力，总时间反而增加。因此必须同时比较寄存器、spill、eligible warps 和 kernel 时间，不能单独追求 occupancy。

8.4 思考题四：Shared Memory 为什么不是 stencil 的必然答案？

Shared memory 只有在跨线程复用足够高、tile 加 halo 后仍保持合理驻留率时才有收益。ABEGPU 的 RHS 同时访问许多场，全部缓存会超过实用容量；只缓存部分 Christoffel 中间量虽减少 spill，却被 shared-memory 访问抵消。还需付出 halo 装载、bank conflict、地址计算和同步成本，因此应先定位重复率最高的字段再设计 tile。

8.5 思考题五：怎样区分合理浮点误差和程序错误？

首先比较完整轨迹 RMS 及其随时间的增长，再比较各 refinement level 的 Hamiltonian 与 momentum constraint，最后检查 launch error、NaN、零 RHS 和轨迹冻结。合法重排造成的误差通常平滑且远低于容差；512-thread 的冻结、FP32 的早期超限和 branchless 的非法访存都属于程序或精度策略失败。

8.6 思考题六：为什么短跑 A/B 可能给出错误结论？

AMR 网格和负载随演化变化，前两步不能代表全程；节点波动也会制造几个百分点的假收益，后期 moving-grid 还会触发新边界。应在同一作业交错 A/B 并比较相同步号，再执行全量检查。per-stream sync 假收益和 branchless 在 step 28 崩溃都说明短跑只能筛选候选。

8.7 思考题七：为什么端到端优化常常不是优化最显眼的 kernel？

评分包含 TwoPuncture、初始化、主演化和分析。ABEGPU 最大单一收益来自解耦 TwoP 构建，CPU 的前中期最大收益来自消除分析 straggler，而收官收益来自 compute_rhs_bssn_ 算法级重写 (-108 s)，都不是继续微调 RHS 的编译细节。前置阶段、最慢 rank 和同步边界会串行进入总时间，因此应按可消除的端到端时间排序，而不是按代码是否容易并行排序。

8.8 思考题八：下一步最值得验证什么？

CPU 热点在重写前收敛到 BSSN 数据移动与 AMR 粗粒度不均衡，继续堆叠编译 flag 的预期收益低；重写后主演化已从约 9.35 s/步降到约 6.98 s/步，OJ 达到 120 分满分，剩余空间主要在更细的导数滚动窗口与边界处理。GPU 仍有 RHS 的 Ricci live set 与依赖 load，以及 prolong3 的大量小任务。下一步应先用 SASS/source correlation 定位 stall，再分别验证按生命周期重构 RHS 和同依赖阶段的 prolong 任务批处理；每次只改变一个变量，并用资源指标、端到端时间和完整 RMS 共同决定是否保留。

9 总结

CPU 路径通过 TwoPuncture、编译 flag、分布式插值与负载均衡，再以 compute_rhs_bssn_ 的算法级重写（点态融合加 k-滚动导数融合）收尾，把 OJ 从 909.42 s、20 分提高到 300.943 s、120 分满分，轨迹 RMS 全程为 0。GPU 路径通过寄存器驻留平衡、强制内联和安全分支消除，正式记录达到 1228.53 s、55 分，后续本地栈进一步接近 1000 s。

失败实验同样构成结论：更多线程、更高 occupancy、更多 shared memory 或更多 stream 都不是普适答案。CPU 侧把“数据依赖地板”误判为不可优化，直到证明编译器调度已最优而算法结构可改；GPU 侧则是逐项排除 live-set、spill 与依赖 load 的伪优化。只有把 profiler 指标还原为数据依赖、访存、负载不均和同步语义，并通过完整物理轨迹验证，性能数字才可信。